08

Part 8 · Cost, Strategy & Interview Toolkit

Spending wisely, avoiding lock-in, migrating, plus a cheat sheet, design template, Q&A and glossary.

- Cost Optimization
- Vendor Lock-in
- Well-Architected Framework
- Migration (6 Rs)
- Cloud for Data Engineers
- AWS / Azure / GCP cheat sheet
- Architecture-answer template
- Rapid-fire Q&A · Glossary

8.1 Cloud Cost Optimization

★★★

IN ONE LINE

Maximising business value per unit of cloud spend through right-sizing, smart purchasing and governance.

Cloud cost optimisation means getting the **most business value for every rupee/dollar spent** — not just spending less. It combines engineering choices, purchase options and governance, practised as **FinOps**.



Figure 8.1a — Typical waste and the lever for each



Figure 8.1b — The FinOps loop

Pricing option	Discount idea	Best for	Catch
On-demand	None (pay per second/hour)	Unpredictable, short-lived	Highest unit price
Reserved / Savings Plan	Substantial discount for 1–3 year commitment	Steady 24×7 workloads	Commitment risk if usage drops
Spot / Preemptible	Very large discount on spare capacity	Fault-tolerant batch, Spark workers, CI	Can be reclaimed with short notice
Serverless	Pay only for execution	Spiky, event-driven	Costly at constant high load

Quick wins

- Delete unused resources and old snapshots.
- Stop non-prod outside working hours.
- Move cold data to cheaper tiers.
- Set a **budget alert** today.

Cost-aware design

- Autoscale instead of over-provisioning.
- Cache and use a CDN to cut egress.
- Use managed services when ops time costs more.
- Compress / partition data to scan less.

DATA ENGINEERING TIE-IN

For warehouses and Spark: auto-suspend idle clusters, use spot workers for batch, partition and compress data, avoid `SELECT *` on pay-per-scan engines, and tag every pipeline's resources so cost can be attributed to teams.

8.2 Vendor Lock-in

★★★

IN ONE LINE

Dependence on one provider's proprietary services that makes switching costly.

Vendor lock-in is when moving away from a provider becomes so costly, slow or risky that you are effectively stuck with it. It comes from proprietary services and APIs, the effort of moving big datasets (**data gravity** and **egress fees**), staff skills and contracts.

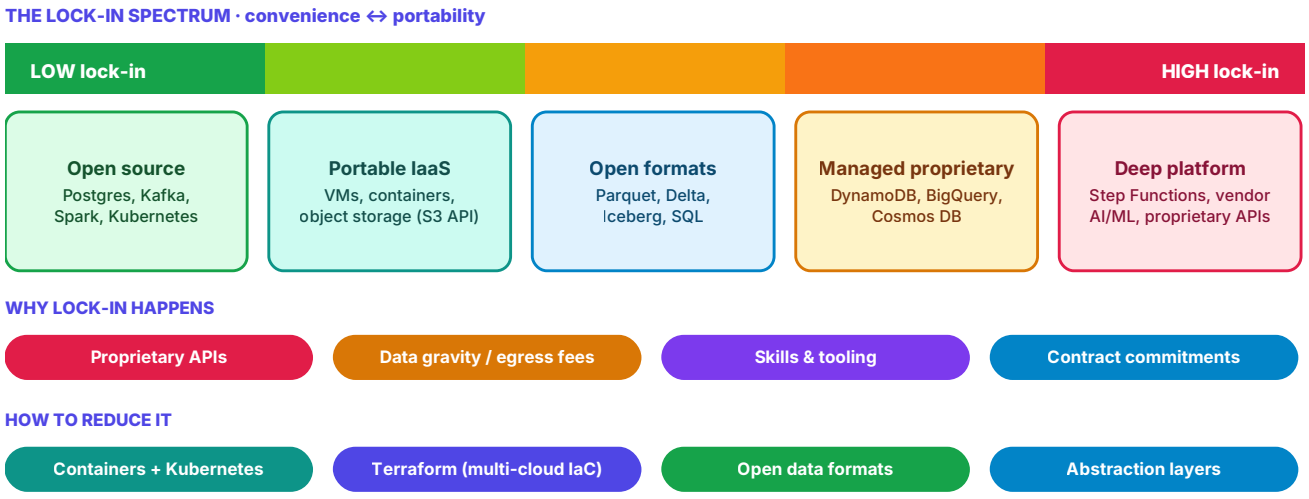


Figure 8.2 — The lock-in spectrum, its causes and mitigations

Strategy	What you do	Trade-off
Use open standards	Postgres, Kafka, Spark, Kubernetes, SQL, Parquet / Delta / Iceberg	May forgo some provider-specific conveniences
Containers & IaC	Docker + Kubernetes + Terraform	Extra abstraction to learn
Multi-cloud	Run on two or more providers	Complexity, cost, lowest-common-denominator features
Exit plan	Document how to migrate; keep data exportable	Planning effort now
Accept it deliberately	Use best-fit managed services; revisit regularly	Needs a clear cost / benefit view

INTERVIEW ANGLE

Balanced answer: **some lock-in is the price of productivity.** Managed services save engineering time, so avoiding all lock-in can cost more than it saves. The art is to **lock in knowingly** where the value is high and keep the **data and core logic portable** (open formats, standard APIs).

COMMON TRAP

"Multi-cloud" is not a free fix. It doubles skills, tooling and security surface. Many companies use **multi-cloud for specific reasons** (best-of-breed, acquisitions, regulation), not as blanket insurance.

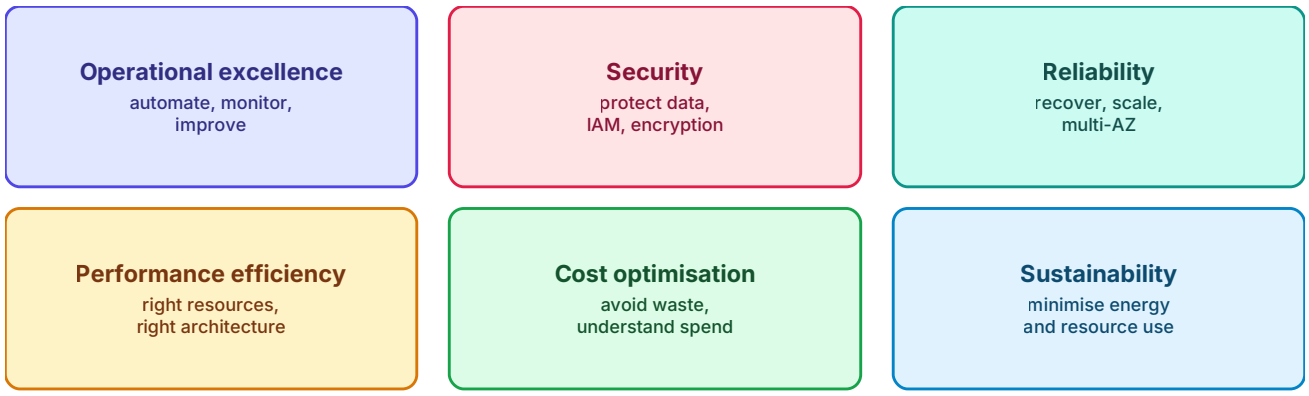
8.3 Well-Architected Framework

★★☆

IN ONE LINE

Six pillars of good cloud design: a checklist for evaluating architectures.

The **Well-Architected Framework** (AWS, with equivalents from Azure and Google) is a set of **best-practice pillars** for judging a cloud design. It gives you a ready-made structure for answering architecture questions and for justifying trade-offs.



AWS names six pillars; Azure and Google Cloud have very similar frameworks.

Use them as a checklist for any architecture discussion.

Figure 8.3 — The six pillars

Pillar	Key question	Where covered here
Operational excellence	Can we run, monitor and improve it?	7.4 Monitoring · 7.5 IaC · 7.6 CI/CD
Security	Is data and access protected?	Part 6 · 1.9 · 4.5
Reliability	Does it recover and scale?	Part 5 · 2.9 · 3.7
Performance efficiency	Are resources matched to need?	2.7 – 2.8 · 7.7 – 7.8
Cost optimisation	Are we spending wisely?	1.4 – 1.5 · 8.1
Sustainability	Are we minimising impact?	Right-sizing, serverless, regions with cleaner energy

INTERVIEW ANGLE

When asked to critique a design, walk the pillars: "security: private subnets and least privilege; reliability: multi-AZ with auto scaling; cost: right-sized and spot for batch…". It shows structure and breadth in under a minute.

8.4 Cloud Migration Strategies (the 6 Rs)

★★☆

IN ONE LINE

Standard options for moving existing applications to the cloud.

Companies rarely build everything new in the cloud: they **migrate existing systems**. The "**6 Rs**" (AWS framing) are the standard choices for each application, ordered roughly by effort and benefit.

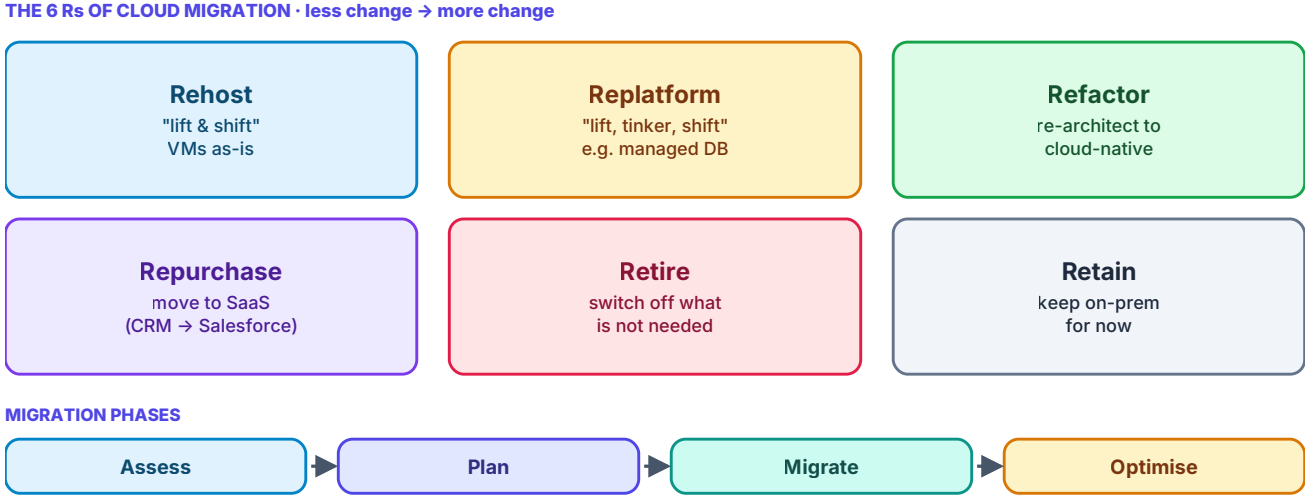


Figure 8.4 — Six migration strategies and the four phases

Strategy	What it means	Example	Effort / benefit
Rehost	Move as-is to cloud VMs ("lift and shift")	App server → EC2	Low / quick win, limited cloud benefit
Replatform	Small optimisations, no core redesign	Self-hosted MySQL → managed RDS	Medium / good balance
Refactor	Re-architect to cloud-native (microservices, serverless)	Monolith → services + functions	High / max benefit
Repurchase	Replace with SaaS	On-prem CRM → SaaS CRM	Medium
Retire	Decommission unneeded systems	Unused reports app	Saves cost
Retain	Leave on-prem for now (compliance, recent investment)	Mainframe	None / revisit later

DATA ENGINEERING TIE-IN

Typical data migration: lift Hadoop/on-prem warehouse data into object storage + a lakehouse (**replatform/refactor**), use CDC to keep source and target in sync during cut-over, validate with row counts and checksums, then retire the old system.

8.5 Cloud for Data Engineering: Putting It Together

★★★

IN ONE LINE

How cloud storage, compute, networking, security and DevOps combine into a data platform.

This is where the two handbooks meet. Every cloud concept so far shows up in a **cloud data platform**: object storage is the lake, VMs / serverless / Kubernetes run the engines, IAM and KMS secure it, networks connect it, and IaC + CI/CD ship it.

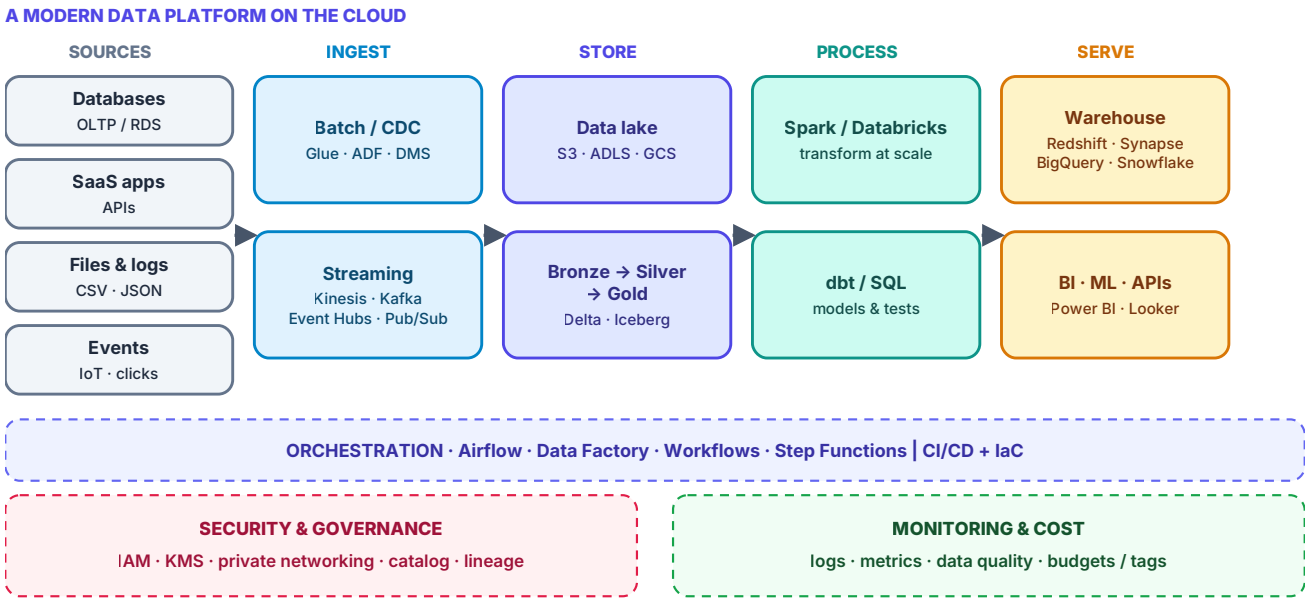


Figure 8.5 — Reference architecture for data engineering on the cloud

Stage	AWS	Azure	Google Cloud
Object storage (lake)	S3	ADLS Gen2 / Blob	Cloud Storage
Batch ETL / integration	Glue, DMS	Data Factory, Synapse pipelines	Dataflow, Data Fusion
Streaming	Kinesis, MSK (Kafka)	Event Hubs	Pub/Sub, Dataflow
Spark / big-data compute	EMR, Glue, Databricks	Databricks, Synapse Spark, HDInsight	Dataproc, Databricks
Warehouse	Redshift	Synapse / Fabric	BigQuery
Orchestration	MWAA (Airflow), Step Functions	Data Factory, Airflow	Cloud Composer (Airflow)
Catalog / governance	Glue Data Catalog, Lake Formation	Purview, Unity Catalog	Dataplex, Data Catalog
BI	QuickSight	Power BI	Looker

INTERVIEW ANGLE

Prepare a 60-second story: *"Data lands from sources into object storage (Bronze), Spark/dbt clean and model it (Silver/Gold) under IAM and encryption, a warehouse or SQL engine serves BI, Airflow orchestrates, Terraform + CI/CD deploy, monitoring and budgets keep it healthy."*

8.6 Multi-Cloud Service Cheat Sheet

★★★

IN ONE LINE

AWS, Azure and Google Cloud equivalents side by side.

The same concepts under different brand names. Learn the **concept** first, then use this table to translate between providers in an interview or on the job.

Concept	AWS	Azure	Google Cloud
Virtual machine	EC2	Virtual Machines	Compute Engine
Auto scaling	Auto Scaling groups	VM Scale Sets	Managed Instance Groups
Containers (managed K8s)	EKS / ECS	AKS	GKE
Serverless functions	Lambda	Azure Functions	Cloud Functions / Cloud Run
Object storage	S3	Blob Storage	Cloud Storage
Block storage	EBS	Managed Disks	Persistent Disk
File storage	EFS / FSx	Azure Files	Filestore
Archive storage	S3 Glacier	Blob Archive tier	Archive storage class
Relational DB (managed)	RDS / Aurora	Azure SQL / Database for PostgreSQL	Cloud SQL / AlloyDB
NoSQL	DynamoDB	Cosmos DB	Firestore / Bigtable
Data warehouse	Redshift	Synapse / Fabric	BigQuery
Virtual network	VPC	VNet	VPC
Firewall rules	Security Groups / NACL	NSG / Azure Firewall	VPC firewall rules
Load balancer	ELB (ALB / NLB)	Load Balancer / App Gateway	Cloud Load Balancing
DNS	Route 53	Azure DNS	Cloud DNS
CDN	CloudFront	Front Door / CDN	Cloud CDN
Private connectivity	Direct Connect	ExpressRoute	Cloud Interconnect
Identity & access	IAM	Entra ID + RBAC	Cloud IAM
Key management	KMS	Key Vault	Cloud KMS
Secrets	Secrets Manager	Key Vault secrets	Secret Manager
Messaging / queues	SQS / SNS	Service Bus / Event Grid	Pub/Sub
Monitoring & logging	CloudWatch / CloudTrail	Azure Monitor	Cloud Monitoring / Logging
IaC (native)	CloudFormation / CDK	Bicep / ARM	Deployment Manager (Terraform common)
CI/CD	CodePipeline / CodeBuild	Azure DevOps / Pipelines	Cloud Build
API management	API Gateway	API Management	API Gateway / Apigee

INTERVIEW ANGLE

Names and bundles change over time (for example Azure Active Directory is now **Microsoft Entra ID**), so confirm exact names in the provider's documentation before quoting them in writing.

8.7 How to Answer a Cloud Architecture Question

★★★

IN ONE LINE

A repeatable structure for designing a hosted application in an interview.

Fresher and junior interviews often end with *"How would you host a website / application on the cloud?"*. Use a structure and tie each choice to a concept from this book.

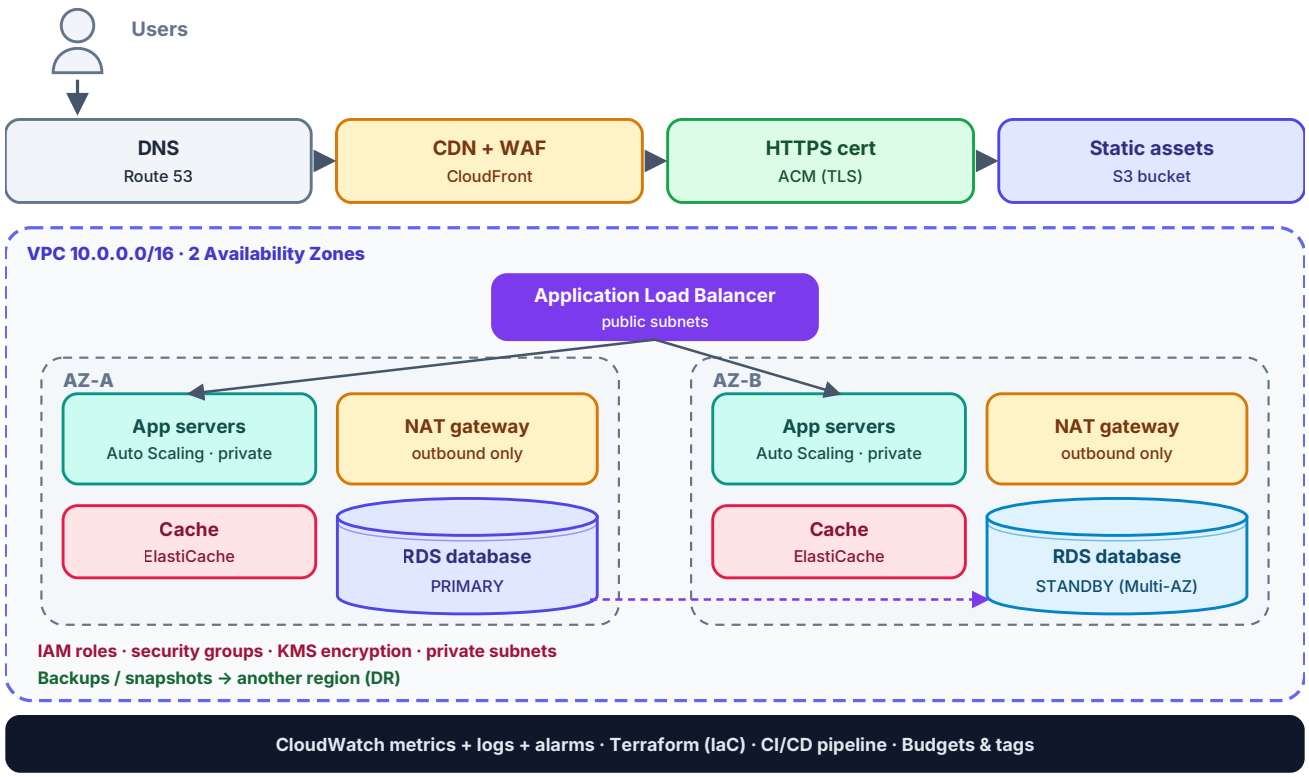


Figure 8.7 — A secure, scalable, highly available three-tier web application

Step	What to say	Concepts
1 Clarify	Users, traffic, latency, data sensitivity, budget, availability target	SLA 5.5 · RTO/RPO 5.7
2 Entry	DNS → CDN + WAF → HTTPS → load balancer	4.2 · 7.8 · 4.3 · 4.8
3 Compute	Stateless app servers in an auto scaling group across 2 AZs (or containers / serverless)	2.9 · 5.1 · 2.4 · 2.6
4 Data	Managed relational DB Multi-AZ, cache for hot reads, object storage for files	3.8 · 3.1 · 5.2
5 Network & security	Public subnet for LB only; private for app + DB; security groups; IAM roles; encryption	4.6 · 4.5 · 6.1 · 6.4
6 Resilience	Backups, cross-region copy, tested recovery plan	3.7 · 5.6
7 Operate	Monitoring, alerts, logs; IaC and CI/CD; tags, budgets, right-sizing	7.4 – 7.6 · 8.1

INTERVIEW ANGLE

Finish with **trade-offs**: "For a tiny site I would use static hosting + serverless to save cost; for global users add multi-region; if the team is small I prefer managed services over self-run VMs." Saying *why* beats naming more services.

8.8 Rapid-Fire Interview Q&A

★★★

IN ONE LINE

Thirty fresher-level questions with answers short enough to say out loud.

Q What is cloud computing?

On-demand delivery of computing resources (servers, storage, databases, networking) over the internet with pay-as-you-go pricing, instead of owning hardware.

Q IaaS vs PaaS vs SaaS?

IaaS gives raw infrastructure (VMs) you manage up from the OS; PaaS gives a platform where you manage only code and data; SaaS is a finished application. More abstraction means less control and less work.

Q Public vs private vs hybrid cloud?

Public: shared provider infrastructure. Private: dedicated to one organisation. Hybrid: on-prem/private connected to public cloud and used together.

Q CapEx vs OpEx?

CapEx is a big upfront purchase of assets; OpEx is recurring spend as you use services. Cloud shifts IT spending from CapEx to OpEx.

Q Explain the shared responsibility model.

The provider secures the cloud infrastructure (security OF the cloud); the customer secures their data, identities, configuration and, depending on the model, OS and applications (security IN the cloud).

Q Virtual machine vs container?

A VM virtualises hardware and includes a full OS (heavy, strong isolation). A container shares the host OS kernel and packages only the app and libraries (lightweight, fast start).

Q What is serverless? Is there really no server?

You run code without managing servers; the provider provisions, scales and bills per execution. Servers still exist but are abstracted away.

Q Scalability vs elasticity?

Scalability: ability to handle growth by adding resources. Elasticity: automatically adding and removing resources as demand changes.

Q Vertical vs horizontal scaling?

Vertical makes one machine bigger (limited, may need downtime). Horizontal adds more machines behind a load balancer (nearly unlimited, resilient, needs stateless design).

Q Object vs block vs file storage?

Object: flat, HTTP-accessed, cheap and scalable (S3). Block: raw fast disk for one server (EBS). File: shared hierarchical network file system (EFS).

Q Durability vs availability?

Durability: data is not lost or corrupted (11 nines for object storage). Availability: the service can be accessed right now (e.g. 99.99%).

Q Is replication a backup?

No. Replication copies changes (including deletions and corruption) almost instantly for availability. Backups keep point-in-time copies you can restore from.

Q Public vs private IP?

Public IPs are internet-routable and globally unique; private IPs are used inside a network and are not routable on the internet. NAT lets private hosts reach out.

Q How does DNS work?

A resolver walks root → TLD → authoritative servers to translate a domain name into an IP, then caches the answer for its TTL.

Q HTTP vs HTTPS?

HTTPS is HTTP over TLS: encrypted, tamper-proof and with server identity via certificates (port 443 vs 80).

Q Security group vs NACL?

Security group: stateful, instance-level, allow rules only. NACL: stateless, subnet-level, allow and deny rules in order.

Q Public vs private subnet?

A public subnet routes to an internet gateway (load balancers). A private subnet has no direct internet route (app servers, databases), using NAT for outbound only.

Q Region vs Availability Zone?

A region is a geographic area; AZs are isolated data-centre groups inside it. Multi-AZ gives high availability; multi-region gives disaster recovery.

Q HA vs fault tolerance?

HA minimises downtime through redundancy and fast failover; fault tolerance continues with no interruption at all and costs more.

Q Define RTO and RPO.

RTO: maximum acceptable downtime. RPO: maximum acceptable data loss, measured as time back from the failure.

Q Authentication vs authorization?

Authentication verifies who you are (401 on failure); authorization decides what you may do (403 on failure).

Q What is least privilege?

Grant only the minimum permissions needed for the shortest time, so mistakes or stolen credentials cause limited damage.

Q Encryption at rest vs in transit?

At rest protects stored data (AES-256, KMS keys); in transit protects data moving over networks (TLS/HTTPS, VPN).

Q Monolith vs microservices?

A monolith is one deployable unit (simple, tightly coupled); microservices are small independently deployable services (scalable, flexible, more operational complexity).

Q Monitoring vs logging?

Monitoring watches metrics and alerts on health; logging records detailed events to explain what happened. Tracing follows a request across services.

Q What is Infrastructure as Code?

Defining infrastructure in version-controlled files (Terraform, CloudFormation) so environments are repeatable, reviewable and automated.

Q CI vs CD?

CI builds and tests every change automatically; continuous delivery keeps software releasable with a manual approval; continuous deployment releases automatically.

Q CDN vs edge computing?

A CDN caches and delivers content from edge locations near users; edge computing runs processing close to devices or users to cut latency.

Q How do you reduce cloud costs?

Right-size, auto-scale, schedule non-prod, use reserved/spot where suitable, tier storage, remove idle resources, tag and set budgets and alerts.

Q What is vendor lock-in and how do you reduce it?

Dependence on proprietary provider services that makes switching hard. Reduce with open standards, containers, Terraform, open data formats and an exit plan, accepting some lock-in for productivity.

8.9 Glossary: Every Term in One Line

★★☆

IN ONE LINE

A quick-revision index of all the concepts in this handbook.

Every term from your list plus the extras, in one place. The last column points to the section with the full explanation and diagram.

Term	One-line definition	§
API	Contract letting software request services from other software.	7.2
Authentication	Proving who you are.	6.2
Authorization	Deciding what an identity may do.	6.2
Auto Scaling	Automatic adding/removing of instances based on demand.	2.9
Availability Zone	Isolated data-centre group within a region.	5.1
Backup	Point-in-time copy for recovery.	3.7
Block storage	Raw fixed-size blocks presented as a disk.	3.2
Bucket	Top-level container for objects in object storage.	3.5
CapEx / OpEx	Upfront asset purchase vs recurring operating cost.	1.4
CDN	Edge cache network delivering content near users.	7.8
CI/CD	Automated build, test and release pipeline.	7.6
Client / Server	Requester and always-on provider of a service.	1.1
Cloud computing	On-demand, metered computing resources over the internet.	1.2
Container	Lightweight packaged app sharing the host OS kernel.	2.4
CSP	Cloud Service Provider (AWS, Azure, GCP).	1.8
Disaster recovery	Restoring systems after a major failure.	5.6
DNS	Translates domain names to IP addresses.	4.2
Durability	Probability data is not lost.	3.6
Edge computing	Processing close to the data source.	7.7
Elasticity	Automatic scale up and down with demand.	2.7
Encryption at rest / in transit	Protecting stored vs moving data.	6.4
Fault tolerance	Continuing with no interruption despite failure.	5.3
File storage	Shared hierarchical network file system.	3.3
Firewall	Allows or blocks traffic by rule.	4.5
High availability	Minimal downtime through redundancy.	5.2
HTTP / HTTPS	Web protocol and its TLS-encrypted version.	4.3
Hybrid cloud	On-prem/private plus public cloud, connected.	1.6
Hypervisor	Software that creates and runs VMs.	2.2

Term	One-line definition	§
IaaS / PaaS / SaaS	Service models by how much the provider manages.	1.7
IaC	Infrastructure defined in code.	7.5
IAM	Controls who can do what on which resources.	6.1
Kubernetes	Container orchestrator.	2.5
Least privilege	Minimum permissions for minimum time.	6.3
Load balancer	Distributes traffic across healthy servers.	4.8
Microservices	Many small independently deployable services.	7.1
Monitoring / Logging	Metrics and alerts vs detailed event records.	7.4
Object storage	Flat, HTTP-accessed storage of objects.	3.1
On-premises	Hardware owned and run by the organisation.	1.3
Pay-as-you-go	Metered billing by actual usage.	1.5
Port / Protocol	Program identifier on a host; rules for communication.	4.4
Private cloud	Infrastructure dedicated to one organisation.	1.6
Public / Private IP	Internet-routable vs internal-only addresses.	4.1
Public cloud	Shared provider-run infrastructure.	1.6
Region	Geographic area containing multiple AZs.	5.1
Replication	Live copy for availability and failover.	3.7
RTO / RPO	Max downtime / max data loss tolerated.	5.7
Scalability	Handling growth by adding resources.	2.7
Serverless	Run code on demand without managing servers.	2.6
Shared responsibility	Provider secures cloud; customer secures use of it.	1.9
SLA / SLO / SLI	Reliability contract, objective and indicator.	5.5
Subnet / VPC	Slice of a private cloud network / the network.	4.6
Vendor lock-in	Costly dependence on one provider.	8.2
Vertical / Horizontal scaling	Bigger machine vs more machines.	2.8
Virtual machine	Software-defined computer with its own OS.	2.3
Virtualization	Splitting one physical machine into many virtual ones.	2.1